

AI Adoption for SDLC — Knowledge Transfer

Date:

Objective: Explain a concrete AI use case in the software development lifecycle, the reasoning framework behind it, and the plan to pilot it safely within two weeks.

Context, Challenges, and Use Case

Business context. Release frequency is increasing while reviewer and QA capacity remains flat. The organization needs faster feedback during development without compromising safety or compliance.

Operational challenges. Reviewers search across pull requests, tickets, architecture decision records, and runbooks to rebuild context. Large diffs can hide critical risks. Test gaps are found late, and postmortems repeat known fixes because relevant knowledge was not surfaced during development.

Use case. *Pull-Request Review and Test Copilot* for service repositories.

Audience. Software engineers and quality engineers.

- **Inputs.** Pull-request diff and metadata, linked tickets, recent failing tests, architecture decision records, runbooks, and code-owner data.
- **Actions.** Identify impacted modules, retrieve similar past changes and incidents, highlight potential risks, draft a concise review with citations, and propose targeted unit tests. A human reviewer approves or edits suggestions before they are adopted.
- **Success criteria.** Reviewer acceptance at or above eighty-five percent, at least two generated tests land on each qualifying pull request, and zero critical policy violations.

Solution Approach

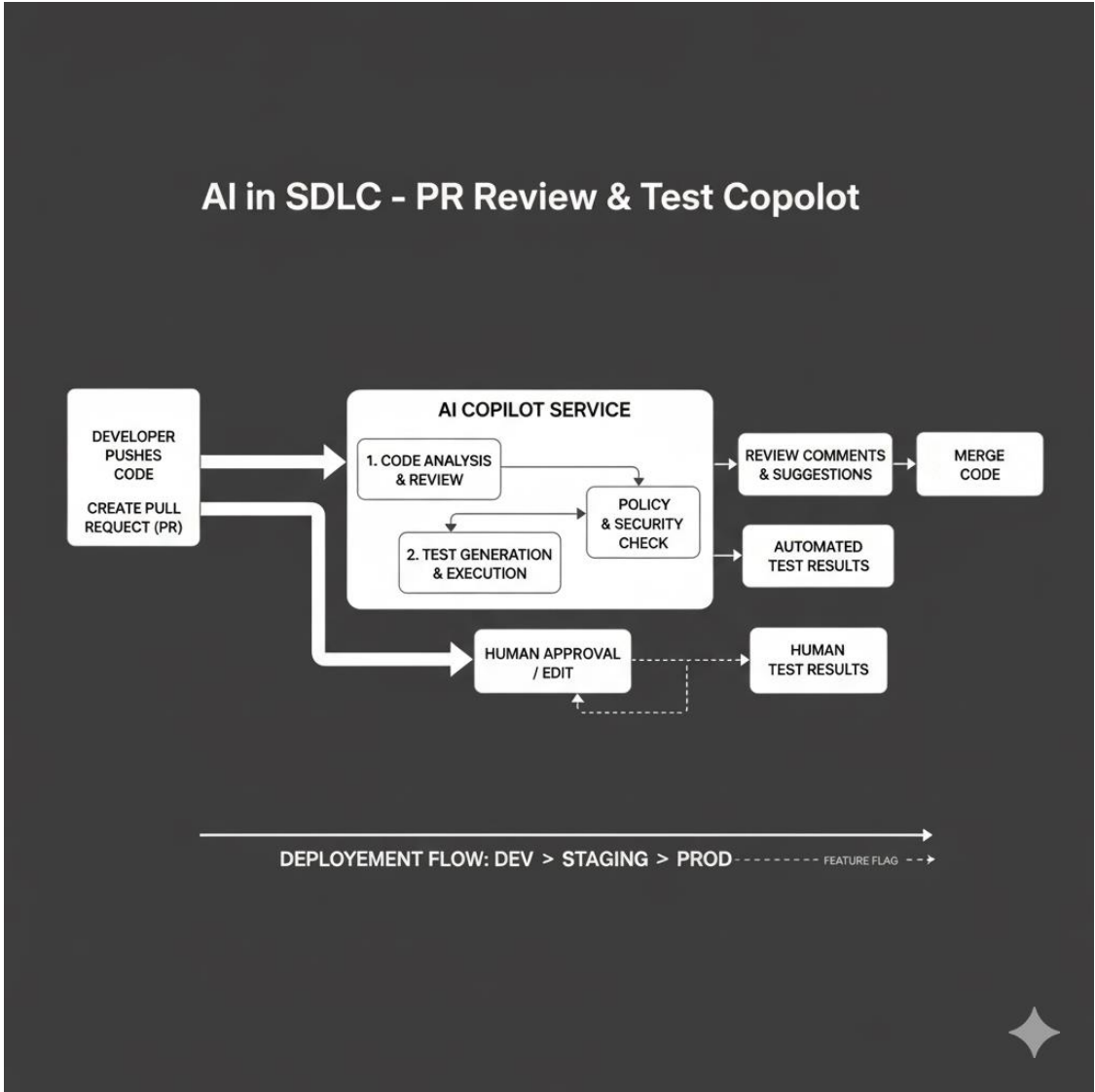
Agentic model. The system follows a cycle of perceive, plan, act, and reflect. It interprets the change, chooses the appropriate tools, executes structured calls, and validates outputs against policy before presenting them to a human reviewer.

Evaluation framework. Outputs are scored on factual grounding, completeness, policy compliance, clarity, and citations. Each criterion is rated from zero to five, aggregated with risk-weighted averages, and compared with regression gates that block releases below threshold.

Implementation plan.

- *Step one: connect systems.* Integrate repository and pull-request APIs, continuous-integration results, the issue tracker, and the knowledge base in read-only mode.
- *Step two: build retrieval.* Index architecture decision records, runbooks, style guides, and prior incident pull requests using hybrid keyword and embedding search.
- *Step three: design policies.* Establish allow and deny lists, add secret and license scanners, and encode organizational rules in policy prompts.
- *Step four: implement the agent.* Implement planning and tool use with strict schemas; generate review bullets and test stubs that reference retrieved evidence.
- *Step five: create the evaluator.* Assemble a golden set of historical pull requests with ground-truth risks and tests; produce automated pass/fail reports.
- *Step six: add observability.* Enable tracing, token and cost telemetry, and a failure taxonomy for tool errors, retrieval misses, and policy blocks.
- *Step seven: pilot and iterate.* Run in shadow mode, compare with the human baseline, adjust prompts and tools, then enable a feature flag on selected repositories.

Solution Diagram



Results

Offline evaluation. Use a golden set of thirty historical pull requests. The system must capture at least two of the top three human-flagged risks for each pull request and propose at least one new test that covers an edge case. Target an overall score of eighty-five percent or higher.

Shadow traffic. Run on live pull requests for one week with an opt-in label. Track reviewer acceptance, false positives, latency, and cost.

Example outcome format.

- Correctness and completeness score: .
- Reviewer acceptance rate: .
- Coverage delta on touched files: .
- Ninety-fifth percentile latency: .
- Cost per run: .
- Critical policy violations: zero.

Next Steps

Timeline. Two-week pilot. Week one connects systems, builds the index, and prepares the golden set. Week two runs the offline evaluation and a shadow trial, then reviews results with security and risk.

Gates. Offline score at or above the threshold, zero critical policy violations, latency within budget, and reviewer acceptance at or above the target.

Ownership. Engineering leads own implementation, product management owns scope and success criteria, and security reviews policies and telemetry.

Reusable Assets

Name: Description: Link

Show time

LIVE DEMO

Thank you!

For more information, contact

Name

Role

<epam>

